

Provenance-Based Context-Enriched Hash Chaining for Secure Audit Logging

S. Sanjay
Department of CSE
Koneru Lakshmaiah Educational
Foundation
Vaddeswaram, Guntur, AP, India.
2300032350cseelge@gmail.com

G. S. M. S. Abhinav
Department of CSE
Koneru Lakshmaiah Educational
Foundation
Vaddeswaram, Guntur, AP, India.
2300031133cseelge@gmail.com

B. Jeswanth
Department of CSE
Koneru Lakshmaiah Educational
Foundation
Vaddeswaram, Guntur, AP, India.
2300030084cseelge@gmail.com

J. Veda Suhas
Department of CSE
Koneru Lakshmaiah Educational
Foundation
Vaddeswaram, Guntur, AP, India.
2300031649cseelge@gmail.com

D. Bhargav Sai
Department of CSE
Koneru Lakshmaiah Educational
Foundation
Vaddeswaram, Guntur, AP, India.
2300032558cseelge@gmail.com

Jonnalagadda Surya Kiran
Department of CSE
Koneru Lakshmaiah Educational
Foundation
Vaddeswaram, Guntur, AP, India.
kiransurya93@kluniversity.in

Abstract

Audit logs are a primary source of security evidence, but an attacker who gains access to a system can modify, delete, insert or reorder log entries to hide their activity. Hash-chained and tree-based logging schemes can reveal that a log has changed, yet they generally do not verify whether an event still belongs to the correct user, session and position in the event sequence. This paper presents a lightweight provenance-based audit logging framework that enriches every event with contextual metadata (user identifier, session identifier, previous event and sequence number) before hashing it with SHA-256 and linking it to its predecessor in a hash chain. Events are grouped into batches summarised by Merkle roots for efficient batch-level verification, and a provenance verification engine checks context, sequence and event relationships together with cryptographic integrity. The framework detects modification, deletion, insertion and reordering of records and reports the outcome as Valid or Tampering Detected through a security dashboard. It needs no blockchain, specialised hardware, kernel-level monitoring or machine learning, and verification runs locally. The design and its working principle are described; a prototype providing authentication, event logging, role-based access and a dashboard is available, with hash-chain integration, provenance verification and Merkle batching under integration.

Keywords

Audit Logging, Tamper Evidence, Hash Chain, Merkle Tree, Provenance Verification, Context Enrichment, SHA-256, Digital Forensics

I. INTRODUCTION

Audit logs are one of the most important forms of security evidence. They record what actually happened on a system (who signed in, which files were opened, which records were changed), and defenders rely on them to reconstruct incidents, establish accountability, satisfy regulatory requirements and support digital forensic investigation. An audit log is useful only if it can be trusted.

That trust is fragile. An attacker who gains access to a system has a strong incentive to modify or delete log entries in

order to hide their tracks. The stored log may still exist and look complete, yet there is no guarantee that its contents are authentic. Audit logs therefore need integrity protection, not just storage. Four kinds of tampering are of particular concern: modification of an existing event, deletion of an important event, insertion of a fake event, and reordering of the original sequence.

Prior work has addressed parts of this problem. Hash-chained logs make each entry depend on its predecessor, so a past entry cannot be silently changed [1]. Encrypted and searchable audit logs combine chaining with confidentiality [2]. Tree-based structures allow large logs to be verified with small proofs [3], and Certificate Transparency applies Merkle trees to publish and verify large batches of records [4]. Provenance-tracking systems record causal links between system events for attack investigation, but often rely on heavyweight information-flow tracking [5]. Other approaches depend on dedicated hardware, blockchain infrastructure or kernel-level monitoring, which are reviewed in Section III. Each of these solves a different part of the problem, and detecting that a log has changed is not the whole of it: an investigator must also know whether an event still belongs to the correct user, session and position in the sequence.

This paper proposes Provenance-Based Context-Enriched Hash Chaining, a lightweight framework that combines four ideas. First, each audit record is enriched with context (user, session, previous event and sequence number), so that an event is stored together with its surroundings. Second, the record and its context are hashed with SHA-256 and linked to the previous entry's hash to form a hash chain. Third, a provenance verification engine checks context, sequence and event relationships together. Fourth, events are grouped into batches whose hashes are summarised by a Merkle root, so that the integrity of a whole batch can be checked efficiently. The hash chain preserves sequence and integrity, provenance captures context and event relationships, and the Merkle tree provides

batch-level integrity; the three work together rather than substituting for one another.

The primary contribution of this work is a design that unifies context-enriched records, hash chaining, provenance verification and Merkle batching in a single practical framework. Unlike blockchain-centered or hardware-backed solutions, it needs no distributed consensus and no dedicated devices; unlike kernel-level monitoring or machine-learning approaches, it stays lightweight, portable and easy to maintain, and its verification runs locally.

II. PROBLEM STATEMENT

Conventional logging is designed for operational needs (reliability, searchability and reasonable performance) rather than for integrity under adversarial conditions. A log stored as ordinary records offers no way to tell whether an entry has been altered after it was written.

We consider an adversary who obtains access to the stored audit records after they have been generated and who tries to conceal activity through one or more of four operations. Modification changes the content of an existing event. Deletion removes an important event. Insertion adds a fake event. Reordering changes the original sequence of events. Hash-based integrity checks can expose many such changes, but detecting that something changed is only part of the requirement. A forensic analyst also needs to know whether each remaining event still belongs to the correct user, the correct session and the correct place in the sequence.

Three gaps follow. First, integrity is not provenance: a hash can show that something changed, but it does not by itself explain how events relate to one another. Second, context is limited: traditional integrity checks may not validate user, session, previous event and sequence together. Third, the alternatives are heavyweight: some approaches depend on blockchain, cloud infrastructure, machine learning, dedicated hardware or kernel-level monitoring, which raises deployment cost. In addition, large collections of logs need an efficient way to verify integrity in batches rather than one event at a time.

Threat model. The adversary can read and rewrite stored log records and may attempt any of the four operations above. Events are assumed to be captured faithfully before they enter the logging pipeline, and the verification code is assumed to be trusted. The adversary is assumed not to regenerate a fully consistent chain and set of batch roots; Section IX-B discusses this residual risk.

The research question motivating this work is therefore: how can audit logs be made tamper-evident while preserving their context, sequence and provenance, using a lightweight design that also supports efficient batch verification?

III. RELATED WORK

The literature offers a range of approaches to secure and tamper-evident logging, each with different trade-offs in

integrity guarantees, context awareness, deployment cost and verification efficiency. We group the most relevant works into foundational schemes (Table I) and more recent systems (Table II).

Schneier and Kelsey [1] showed that hash chains let a log resist silent modification, although the scheme detects changes in content and not in user or session context. Waters et al. [2] combined hash chaining with encryption to obtain searchable, tamper-evident database logs, with the emphasis on confidentiality and search. Crosby and Wallach [3] introduced tree-based structures that allow large logs to be checked with compact proofs, and Laurie et al. [4] used Merkle trees for Certificate Transparency; both are strong on structural integrity but do not model the relationships between events. Ji et al. [5] tracked provenance and causal links between system events for attack investigation, at the cost of heavyweight information-flow tracking.

More recent work extends these ideas in several directions. The Reverse File System [6] offers low-cost tamper-evident WORM storage for logs, and Aßmuth et al. [7] keep encrypted audit records in an immutable database using secret sharing; both concentrate on storage and privacy rather than on relationships between events. ForensiBlock [8] combines blockchain, provenance and Merkle roots to protect digital forensic evidence, and the Entangled Merkle Forest [9] supports integrity and version control for cloud files; neither addresses real-time system-event logging with user, session and process context. SealFSv2 [10] pairs storage-based protection with cryptographic ratcheting, while HARDLOG [11] protects logs with a dedicated audit device; both give strong integrity but lack rich event context, and HARDLOG needs additional hardware. LogKernel [12] turns audit logs into behaviour provenance graphs for threat hunting, yet it assumes the logs are already trustworthy and does not cryptographically protect their event sequence. Finally, SealFS [13] protects Linux logs against modification and deletion through storage-based authentication, with limited analysis of context or provenance.

Taken together, these works show that hash chaining, tree-based verification, storage protection and provenance tracking can each be effective on their own. Traditional logging is simple but offers no integrity protection. Hash chains detect modification and deletion but do not capture user, session or event relationships. Merkle verification checks batches efficiently but not event-level provenance. Provenance systems track origin and relationships but are often heavyweight. Blockchain and external anchoring add an independent reference at the cost of extra infrastructure, and kernel-level monitoring captures detailed activity but is resource-heavy and complex to deploy. What remains less addressed is a lightweight framework that combines context, sequence, provenance and efficient batch verification in one design, which is the gap this paper targets.

TABLE I
Comparative Survey of Foundational Work on Tamper-Evident and Provenance-Aware Audit Logging

Ref.	Authors	Year	Framework / Method	Key Contribution	Limitation	Relevance to This Work
[1]	Schneier & Kelsey	1999	Hash-chained secure audit logs	Makes each log entry depend on its predecessors so that a log cannot be silently changed without detection	Detects change in content, but not user/session context or event relationships	Basis of the SHA-256 hash chain; extended here with context inside every record
[2]	Waters et al.	2004	Encrypted and searchable audit log	Combines hash chaining with encryption for searchable, tamper-evident database logs	Focused on confidentiality and search, not batch verification or provenance	Shows chaining over database-resident records; provenance is left unaddressed
[3]	Crosby & Wallach	2009	Tree-based tamper-evident history	Uses a tree structure so that large logs can be verified with small, efficient proofs	Strong on structural integrity; does not model contextual event relationships	Motivates Merkle batch verification with logarithmic-size proofs
[4]	Laurie et al.	2013	Certificate Transparency (RFC 6962)	Uses Merkle trees to publish and verify large batches of records efficiently	Built for public certificate logs, not general application-level audit events	Validates Merkle batching; applied here to context-enriched audit events
[5]	Ji et al.	2017	RAIN: refinable attack investigation	Tracks provenance and causal links between system events for attack analysis	Relies on heavyweight information-flow tracking; not lightweight	Motivates provenance; replaced here by a small set of lightweight context fields

TABLE II
Comparative Survey of Recent Work on Secure Audit Logging and Log Provenance

Ref.	Authors	Year	Framework / Method	Key Contribution	Limitation	Relevance to This Work
[6]	Guardiola M. et al.	2025	Reverse File System (Socarrat)	Low-cost tamper-evident WORM storage for logs using an external USB device	Focuses on storage protection, not contextual provenance	Storage-level protection complements record-level context verification
[7]	Aßmuth et al.	2024	Encrypted audit trail in an immutable database	Secure, privacy-preserving and immutable audit logging using secret sharing	Focuses mainly on privacy and storage, not event relationships	Shows immutability and privacy; this work targets event-level context
[8]	Akbarfam et al.	2023	ForensiBlock: provenance-driven blockchain	Uses blockchain, provenance and Merkle roots to secure digital forensic evidence	Aimed at forensic evidence, not real-time system-event logging; needs blockchain	Confirms the value of provenance with Merkle roots, without blockchain overhead
[9]	Bappy et al.	2023	Entangled Merkle Forest	Cloud data integrity and file-version auditing with node-sharing Merkle trees	Focuses on cloud files; no user/session/process provenance	Supports Merkle structures for efficient integrity checks
[10]	Guardiola M. & Soriano-Salvador	2023	SealFSv2	Combines storage-based protection with cryptographic ratcheting / hash chaining	Strong integrity protection but does not verify rich event context	Baseline for chained integrity; context verification is added here
[11]	Ahmad et al.	2022	HARDLOG	Protects audit logs with a dedicated hardware audit device	Protects logs but lacks rich context and provenance; needs extra hardware	Contrasts hardware-dependent with software-only protection
[12]	Li et al.	2022	LogKernel	Converts audit logs into behaviour provenance graphs for threat hunting	Provenance analysis without cryptographic protection of the event sequence	Analysis presupposes trustworthy logs, which this framework helps to supply
[13]	Soriano-Salvador & Guardiola M.	2021	SealFS	Protects Linux logs against modification and deletion using storage-based authentication	Strong tamper protection but limited context/provenance analysis	Baseline for tamper-evident storage; contrasts with record-level context

IV. PROPOSED SYSTEM

The proposed framework integrates context-enriched event records with cryptographic hash chaining, provenance verification and Merkle-tree batch verification. Rather than treating these as independent mechanisms, the design connects them into a pipeline in which each stage feeds the next and each stage’s output can be checked independently of the others.

The pipeline operates as follows:

Event Capture: A security event (a login, file access, database access or similar) is detected and recorded as it happens.

Context Enrichment: The user, session, previous event and sequence number are attached to the event, so that it is stored together with its context.

SHA-256 Hash Computation: A hash is generated from the event plus its context.

Hash Chain Linking: The new hash incorporates the previous event’s hash, preserving chronological order in a verifiable way.

Batch Formation: Events are grouped into batches for verification.

Merkle Tree Construction: A tree of hashes is built over the batch, and its root serves as a compact integrity summary of the whole batch.

Local Storage: Records, hashes and batch data are stored in a local database.

Provenance Verification: The verification engine checks context, sequence and the relationships between events.

Tamper Detection: Any mismatch is flagged, and the result is reported as Valid or Tampering Detected.

Three roles are deliberately kept distinct: the hash chain preserves event sequence, provenance verifies context and event relationships, and the Merkle tree provides batch integrity. The core design objective is that an attacker who alters historical records cannot do so without leaving detectable evidence, not only in the content of the log but also in its sequence, its context and the relationships between its events.

V. SYSTEM ARCHITECTURE

The architecture is organised into five functional layers, each with a distinct responsibility. Together they form a chain of accountability from a raw system event through to a verification result.

A. Event Capture Layer

This layer detects security events at the point where they originate (user or system activity such as logins, file actions and database actions) and records each one. Events are captured at application level, so the design needs no kernel modification or privileged monitoring agent and remains portable across platforms.

B. Context Enrichment Layer

Each captured event is enriched with the user identifier, session identifier, previous event and sequence number. This turns an isolated entry such as “Open File” into a record that states who performed it, in which session, after which event and at which position in the sequence. The enriched record is serialised into a canonical form suitable for hashing.

C. Integrity Layer

This layer computes a SHA-256 hash of each enriched record and links it to the previous entry’s hash. Because every chain value depends on all earlier ones, altering an earlier record invalidates every later value, so the content and chronological order of events cannot be changed silently.

D. Merkle Batch and Storage Layer

Entries are grouped into batches, and the hashes of each batch are organised into a Merkle tree whose root represents the whole batch. Records, hashes and batch data are kept in a local database. Checking any single record against its batch requires only a logarithmic number of sibling hashes rather than the entire batch.

E. Provenance Verification and Response Layer

This layer periodically recomputes the hash chain and Merkle roots from the stored records and checks context, sequence and relationships. Any discrepancy, whether from modification, deletion, insertion or reordering, surfaces as a mismatch. A security dashboard displays the logs and verification status, and the engine reports Valid or Tampering Detected together with the batch identifier and the first record at which verification failed.

Fig. 1 illustrates the complete flow across all five layers, from user or system activity through context enrichment, hashing, chaining and Merkle batching to the verification result. Fig. 2 shows the hash chain and Merkle batch structure, using the deletion of an event as a worked example.

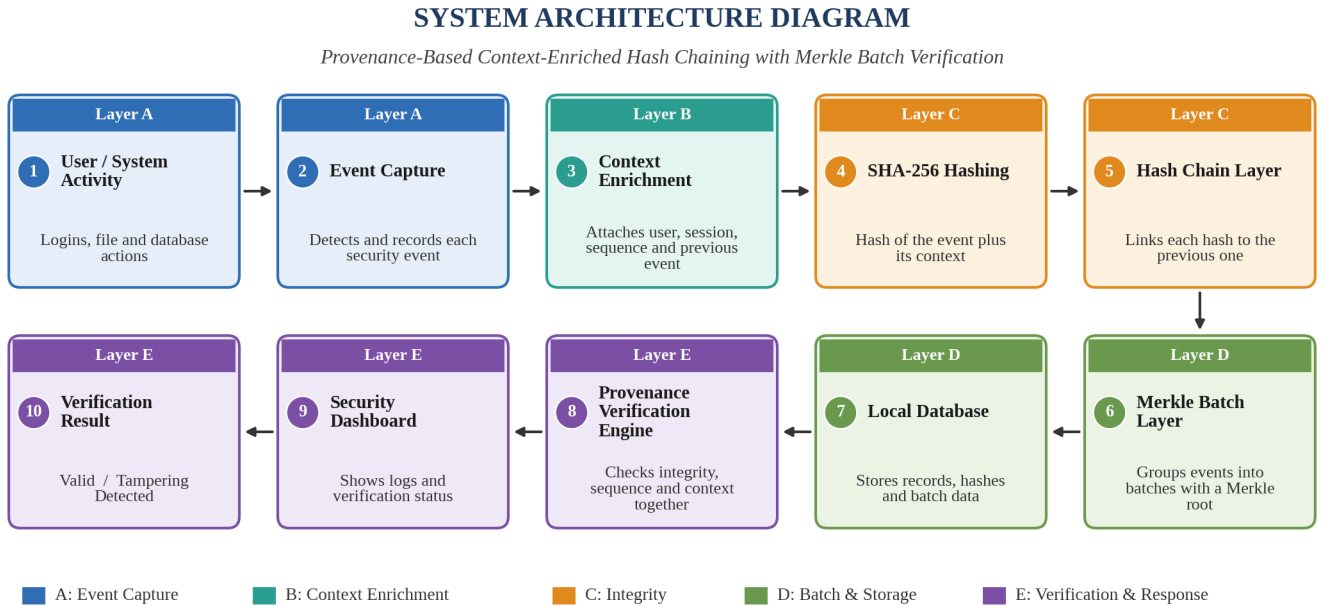
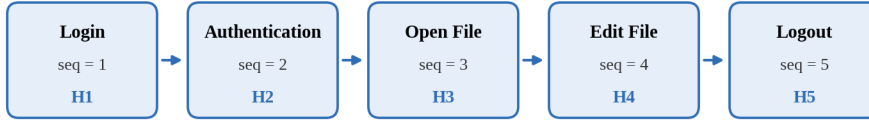


Fig. 1. System Architecture of Provenance-Based Context-Enriched Hash Chaining: End-to-End Flow from Event Capture Through Context Enrichment, SHA-256 Hashing, Hash Chaining, Merkle Batching and Provenance Verification to the Verification Result

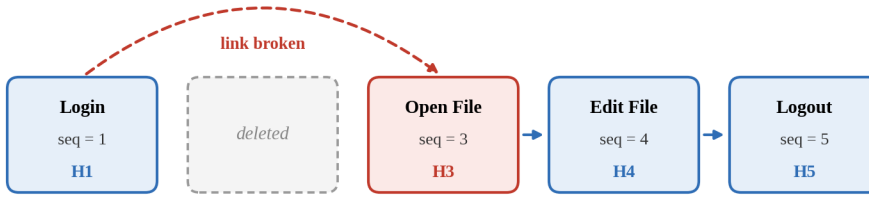
(a) Hash chain with contextual provenance

Normal sequence



$$H(n) = \text{SHA-256}(\text{event} \mid \text{context} \mid H(n-1))$$

After attacker deletes "Authentication"



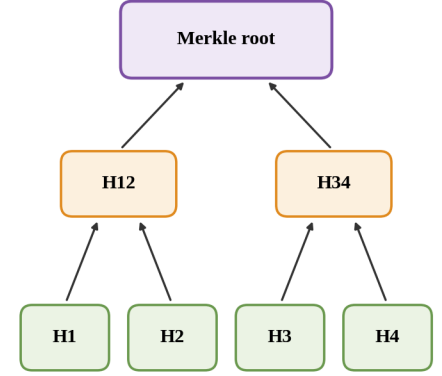
Checks that fail at "Open File":

Hash chain: stored previous hash $H2 \neq$ recomputed $H1$

Sequence: $1 \rightarrow 3$ (gap) Previous event: "Authentication" $\neq$ "Login"

Merkle batch: recomputed root $\neq$ stored root $\rightarrow$ **TAMPERING DETECTED**

(b) Merkle-tree batch verification



$H1-H4$ = event hashes (chain values)

Parent = $\text{SHA-256}(\text{left} \mid \text{right})$

Any modified, missing, inserted or reordered leaf changes the root, so a single comparison checks the batch.

Inclusion proof: $O(\log m)$ sibling hashes for a batch of m events.

Fig. 2. (a) Hash Chain With Contextual Provenance: Normal Sequence and the Effect of Deleting the "Authentication" Event; (b) Merkle-Tree Batch Verification

VI. IMPLEMENTATION DETAILS

The framework is being realised as an extension of an existing application prototype. The completed part of the prototype provides user authentication, security event logging, user and administrator roles, a dashboard, and monitoring of IP addresses and security events. The context-enriched event records, hash-chain integration, provenance verification and Merkle batching described below are under integration.

A. Event Capture and Context Enrichment

Events of interest include login, authentication, file access, file edits, database access, logout and IP-related security events. For every event the application attaches the context fields listed in Table III before hashing. A canonical, deterministic serialisation is essential: the same logical record must always produce identical bytes, otherwise verification would raise false mismatch alerts.

TABLE III
Context-Enriched Audit Record (Example)

Field	Purpose	Example
User ID	Who performed the event	U101
Session ID	Session the event belongs to	S5001
Current Event	What happened	Open File
Previous Event	What happened before	Authentication
Sequence No.	Position within the session	3
Timestamp	When it happened	10:15:32
Previous Hash	Link to preceding chain value	abc123...

B. Hash Chaining

For the n -th record, let E_n be the event and $C_n = (u_n, s_n, e_{n-1}, q_n, t_n)$ its context: user, session, previous event, sequence number and timestamp. The system computes

$$H_n = \text{SHA-256}(E_n \parallel C_n \parallel H_{n-1}) \quad (1)$$

where H_{n-1} is the hash of the preceding entry and $\parallel$ denotes byte concatenation [14]. The first entry uses a fixed genesis value H_0 as its predecessor hash. For the sequence Login, Authentication, Open File this gives $H_1 = \text{SHA-256}(\text{Login} \parallel C_1 \parallel H_0)$, $H_2 = \text{SHA-256}(\text{Authentication} \parallel C_2 \parallel H_1)$ and so on. Because the context is part of the hashed input, changing the user or session of a record alters its hash just as changing the event itself does. Any modification to an earlier entry changes its hash, which then changes every subsequent chain value.

C. Provenance Verification

A hash shows that a record changed; it does not show that an unchanged record still belongs where it is. The provenance verification engine therefore asks five questions of every record. The hash chain runs across the whole log stream, whereas sequence numbers and previous events are tracked per session.

Who? The user recorded in the entry must match the user who owns the session, as established by the authentication event.

Which session? The session identifier must refer to a valid session of that user, and the event must fall within its lifetime.

What happened before? The previous-event field must equal the actual preceding event of the same session.

What sequence number? The sequence number must be exactly one greater than that of the preceding event in the session, so gaps and duplicates become visible.

Does this event belong here? The pair (previous event, current event) must be an allowed transition, for example Authentication before Open File. The set of allowed transitions is defined per application.

A record is accepted only when its chain value and all five provenance checks succeed.

D. Merkle Batch Verification

For a batch of m entries, each entry hash H_n forms a leaf of a Merkle tree [15], and internal nodes are computed by hashing adjacent children upward:

$$M_{i,j} = \text{SHA-256}(M_{i+1,2j} \parallel M_{i+1,2j+1}) \quad (2)$$

The root $M_{0,0}$ compresses the integrity of the entire batch; when a level has an odd number of nodes the last node is duplicated. The root is stored with the batch identifier, the range of entries covered and a timestamp. Verifying that a single record belongs to a batch requires only $O(\log m)$ sibling hashes, and verifying the batch as a whole requires one root comparison. Because the chain continues across batch boundaries, removing an entire batch also breaks the chain.

E. Local Storage and Verification Workflow

Records, hashes and batch data are kept in a local database, and verification runs locally without any external service. The engine reads the stored records in order, recomputes each chain value, re-applies the provenance checks and rebuilds the Merkle root of each batch. If every comparison succeeds the log is reported as Valid; otherwise the engine reports Tampering Detected together with the batch identifier, the index of the first failing record and the check that failed. Results are shown on the security dashboard.

F. Tamper Detection

Table IV summarises how each tampering operation is exposed. Fig. 2(a) walks through the example used throughout the design: an attacker deletes the “Authentication” event from the sequence Login, Authentication, Open File, Edit File, Logout. The hash chain no longer matches at “Open File”, whose stored previous hash refers to the removed entry; the sequence number jumps from 1 to 3; the previous-event field (“Authentication”) does not match the actual predecessor (“Login”); and the transition from Login to Open File is not allowed. The recomputed Merkle root also differs from the stored root, so the batch is flagged.

TABLE IV
How Each Tampering Operation Is Exposed

Operation	Detecting checks
Modification	Recomputed entry hash differs from the stored hash; every later chain value breaks; altered context fails the provenance checks; batch root changes.
Deletion	Sequence gap; the next entry’s previous-hash and previous-event fields do not match its actual predecessor; leaf count and batch root change.
Insertion	Chain breaks at the successor; duplicate or out-of-range sequence number; invalid transition or session binding; batch root changes.
Reordering	Sequence numbers and timestamps out of order; chain links fail; leaf order changes the batch root.
Context forgery	User and session fields are inside the hashed input, so the hash mismatches; the user-session binding check also fails.
Full rewrite of chain and roots	Not detected by local verification alone; requires keyed hashing, signatures or an external anchor (Section IX-B).

VII. EXPERIMENTAL SETUP

The evaluation is planned around the application prototype with a local database running on a commodity machine, whose specification will be reported together with the results. Three scenarios are considered.

Normal operation: generation of representative event sequences (logins, authentication, file access, edits and logouts) for multiple users and concurrent sessions, followed by verification of the untampered log. This scenario measures the false-alarm rate, which should be zero.

Tampering scenarios: deliberate modification, deletion, insertion and reordering of stored records, and alteration of context fields such as user and session identifiers, after generation, each followed by a verification run to confirm detection and to locate the first affected record.

Scale scenario: increasing batch sizes and log volumes, to assess verification time and the per-record storage overhead of the added context and hash fields.

The metrics of primary interest are the detection rate for each tampering type, the false-alarm rate on untampered logs, the accuracy of locating the first affected record, verification time per batch, and storage overhead per record.

VIII. EXPECTED OUTCOMES AND ANALYSIS

Because the prototype is still being integrated, this section analyses the behaviour that follows from the design and identifies what the planned experiments are meant to confirm. No measured performance figures are claimed at this stage.

The framework is expected to deliver tamper-evident audit logs, context-aware records, sequence verification, provenance verification, detection of missing or unexpected events, efficient batch verification and clear tampering alerts.

Detection coverage. Modification changes an entry hash and every later chain value. Deletion and insertion break the previous-hash link and the sequence numbering at the affected point, and reordering violates sequence, timestamp and chain order. Each of these is additionally reflected in the batch root.

Because context fields are hashed, tampering with user or session identity is exposed in the same way (Table IV). Detection thus rests on several independent checks, and an attacker must defeat all of them rather than one.

Efficiency. Full verification is linear in the number of entries, since each is rehashed once and its provenance checks are constant-time field comparisons. Building a Merkle tree over a batch of m entries adds $O(m)$ hash operations, after which a single-record membership check needs only $O(\log m)$ hashes. Only SHA-256 and simple comparisons are involved, so no specialised hardware is needed.

Comparison with alternatives. Compared with hash-chain-only schemes such as [1], the framework additionally verifies context and event relationships. Compared with tree-based schemes [3], [4], it adds event-level provenance. Compared with provenance systems based on information-flow tracking [5] or graph analysis [12], it stays lightweight by recording a small fixed set of context fields instead of tracking causal flows, and it protects the event sequence cryptographically. Compared with blockchain-based [8], hardware-based [11] and storage-based [10], [13] designs, it needs no extra infrastructure; the price is the weaker trust model discussed in Section IX.

IX. ADVANTAGES AND LIMITATIONS

A. Advantages

The framework offers several practical benefits. It makes audit logs context-aware, so that each event carries who performed it, in which session, what came before and where it sits in the sequence. It detects modification, deletion, insertion and reordering through independent checks that reinforce one another. It supports efficient batch verification through Merkle roots. It is lightweight, needing no blockchain, specialised hardware, kernel-level monitoring or machine learning. Verification runs locally, and the design is portable and maintainable because it relies only on standard hashing and a local database.

B. Limitations

Several limitations should be acknowledged. First, the framework is tamper-evident rather than tamper-proof: it detects changes but cannot prevent them. Second, because chain values and Merkle roots are stored locally, an adversary with write access to all stored data who can recompute the entire chain and every batch root would produce a log that verifies as Valid. Closing this gap requires a trust anchor outside the attacker's reach, such as keyed hashing, digital signatures over batch roots or publication of roots to an external store; these add infrastructure and are left to future work. Third, records altered before they enter the pipeline are not detected, and the context is only as reliable as the application that supplies it. Fourth, the allowed-transition rules used by the provenance check must be defined for each application, and overly strict rules could raise false alarms. Finally, the framework is not yet fully implemented, and its performance must be validated through repeated measurements on varied workloads.

X. FUTURE WORK

The immediate next steps are to complete the verification engine, to test the four tampering scenarios on the prototype, and to benchmark verification on larger event volumes. To close the local-verification gap, keyed hashes (HMAC) or digital signatures over batch roots could be introduced, with optional external anchoring of roots to append-only or WORM storage for high-assurance deployments. In multi-host environments, per-host Merkle roots could be aggregated into cross-host proofs. Once integrity is established, the verified and context-rich event stream could support provenance-aware anomaly detection. Finally, formal analysis of the combined chain, provenance and Merkle checks under realistic adversary models would strengthen confidence in real-world use.

XI. CONCLUSION

This paper presented Provenance-Based Context-Enriched Hash Chaining, a lightweight framework for tamper-evident audit logging that combines context-enriched records, SHA-256 hash chains, provenance verification and Merkle-tree batching. It addresses a basic weakness of conventional logging: once an attacker has access, the entries that would reveal their actions can be altered, removed, added or reordered, and a bare integrity check says little about whether the surviving events still belong to the right user, session and sequence.

By storing each event with its context, chaining it to its predecessor, verifying provenance alongside integrity and summarising batches with Merkle roots, the framework detects modification, deletion, insertion and reordering and reports the outcome as Valid or Tampering Detected, without blockchain, specialised hardware, kernel-level monitoring or machine learning. It is well suited to settings where accountability and the trustworthiness of audit evidence matter, such as security monitoring, regulated environments and post-incident investigation. With the verification engine and tampering tests still to be completed, the framework provides a foundation for future extensions toward keyed or externally anchored verification, distributed deployment and formal security analysis.

REFERENCES

- [1] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999.
- [2] B. R. Waters, D. Balfanz, G. Durfee, and D. K. Smetters, "Building an encrypted and searchable audit log," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA, 2004.
- [3] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Secur. Symp.*, Montreal, QC, Canada, 2009.
- [4] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," IETF, RFC 6962, Jun. 2013.
- [5] Y. Ji *et al.*, "RAIN: Refinable attack investigation with on-demand inter-process information flow tracking," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Dallas, TX, USA, 2017.
- [6] G. Guardiola Múzquiz, J. González-Gómez, and E. Soriano-Salvador, "The Reverse File System: Towards open cost-effective secure WORM storage devices for logging," arXiv:2509.17969, 2025.

- [7] A. Aßmuth, R. Duncan, S. Liebl, and M. Söllner, "A secure and privacy-friendly logging scheme," arXiv:2405.11341, 2024.
- [8] A. Jodeiri Akbarfam, M. Heidaripour, H. Maleki, G. Dorai, and G. Agrawal, "ForensiBlock: A provenance-driven blockchain framework for data forensics and auditability," in *Proc. IEEE Int. Conf. Trust, Privacy Secur. Intell. Syst. Appl. (TPS-ISA)*, 2023, pp. 136–145.
- [9] F. H. Bappy *et al.*, "Towards immutability: A secure and efficient auditing framework for cloud supporting data integrity and file version control," arXiv:2308.04453, 2023.
- [10] G. Guardiola Múzquiz and E. Soriano-Salvador, "SealFSv2: Combining storage-based and ratcheting for tamper-evident logging," *Int. J. Inf. Secur.*, vol. 22, no. 2, pp. 447–466, 2023.
- [11] A. Ahmad, S. Lee, and M. Peinado, "HARDLOG: Practical tamper-proof system auditing using a novel audit device," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 1791–1807.
- [12] J. Li, R. Zhang, J. Liu, and G. Liu, "LogKernel: A threat hunting approach based on behaviour provenance graph and graph kernel clustering," *Secur. Commun. Netw.*, 2022.
- [13] E. Soriano-Salvador and G. Guardiola-Múzquiz, "SealFS: Storage-based tamper-evident logging," *Comput. Secur.*, vol. 108, 2021.
- [14] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.
- [15] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology—CRYPTO '87*, LNCS 293. Berlin, Germany: Springer, 1988, pp. 369–378.